

2주차 예습과제

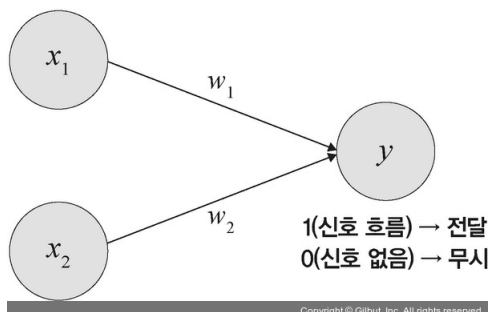
■ 과제 종류	예습과제
■ 범위	딥러닝 파이토치 교과서 4장

Chapter 04. 딥러닝 시작

4.1 인공 신경망의 한계와 딥러닝 출현

| 퍼셉트론

- 프랭크 로젠블라트(Frank Rosenblatt)가 1957년에 고안한 선형 분류기
- 오늘날 인공 신경망에서 이용하는 구조(입력층, 출력층, 가중치로 구성된 구조)
- 흐름이 있는 다수의 신호를 입력 받아 하나의 신호를 출력하는데, 이 신호를 입력을 받아 '흐른다/안 흐른다(1 or 0)'는 정보를 앞으로 전달하는 원리로 작동



- AND 게이트: 모든 입력이 '1' 일 때 작동
- OR 게이트: 입력 중 둘 중 하나만 '1' 이거나 둘 다 '1'일 때 작동
- XOR 게이트: 입력 중 한 개만 '1' 일 때 작동

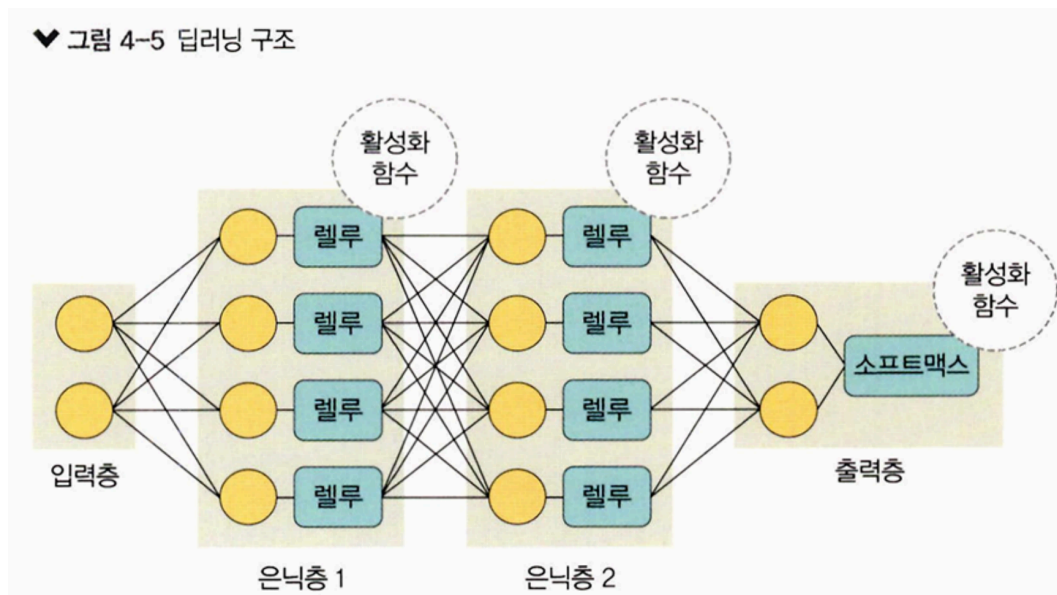
⇒ XOR 게이트는 데이터가 비선형적으로 분리되어 제대로 된 분류가 어려움. 즉, 단층 퍼셉트론에서는 AND, OR 연산은 학습이 가능하지만, XOR 학습 불가

⇒ 입력층과 출력층 사이의 중간층(은닉층)을 두어 극복. 은닉층이 있어 비선형적으로 분리 되는 데이터에 대해서도 학습이 가능함. ⇒ **다층 퍼셉트론**

- **심층 신경망(DNN):** 입력층과 출력층 사이에 은닉층이 여러 개 있는 신경망 (=딥러닝)

4.2 딥러닝 구조

4.2.1 딥러닝 용어



- 입력층, 출력층, 두 개 이상의 은닉층으로 구성됨.

딥러닝 구성 요소

구분	구성 요소	설명
층	입력층(input layer)	데이터를 받아들이는 층
	은닉층(hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층(output layer)	신경망의 최종 결괏값이 포함된 층

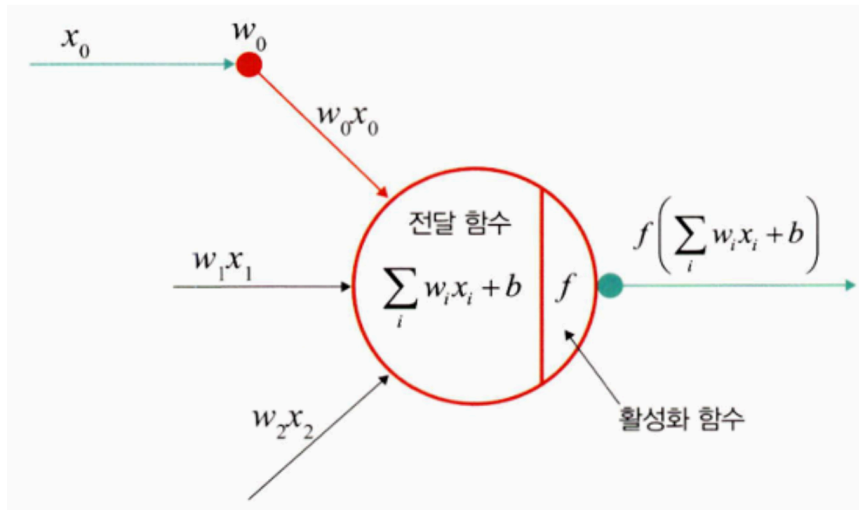
구분	구성 요소	설명
가중치(weight)		노드와 노드 간 연결 강도
바이어스(bias)		가중합에 더해 주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
가중합(weighted sum), 전달 함수		가중치와 신호의 곱을 합한 것
함수	활성화 함수(activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
	손실 함수(loss function)	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

가중치

- 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소

가중합 또는 전달 함수

- 각 노드에서 들어오는 신호에 가중치를 곱해 다음 노드로 전달하는데, 이 값들을 모두 더한 합계를 가중합이라고 한다.
- 또한, 노드의 가중합이 계산되면 이 가중합을 활성화 함수로 보내기 때문에 전달함수 (transfer function)라고도 한다.



- 가중합 구하는 공식 (w: 가중치, b: 바이어스)

$$\sum_{i=0}^n w_i x_i + b$$

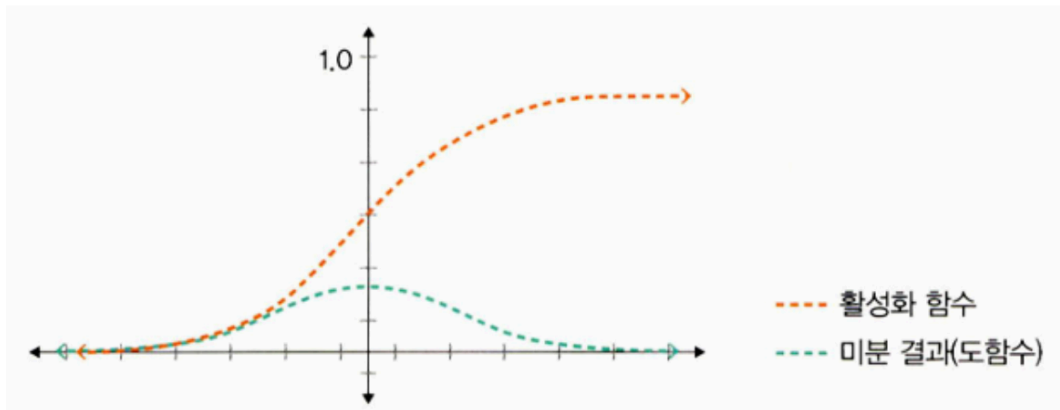
활성화 함수

- 전달 함수에서 전달 받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수

1 시그모이드 함수

- 선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형
- 주로 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현하는데 사용한다.
- 과거에는 인기가 많았으나, '기울기 소멸 문제(vanishing gradient problem)'가 발생하여 딥러닝 모델에서 잘 사용하지 않는 추세

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

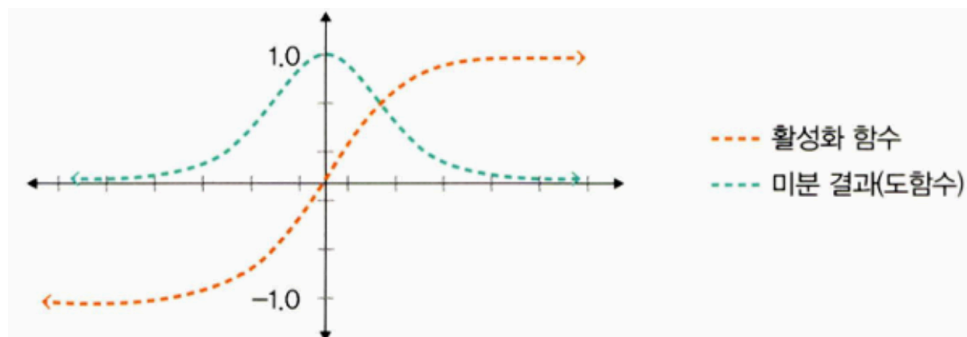


시그모이드 활성화 함수와 미분 결과

2 하이퍼볼릭 탄젠트 함수

- 선형 함수의 결과를 -1~1 사이에서 비선형 형태로 변형
- 시그모이드에서 결괏값의 평균이 0이 아닌 양수로 편향된 문제를 해결하는데 사용했지만,
기울기 소멸 문제 발생 해결 ❌

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



하이퍼볼릭 탄젠트 활성화 함수와 미분 결과

⇒ 시그모이드 함수와 비교

📌 학습 효율성과 기울기 소실

- 시그모이드 함수는 출력이 항상 양수(0~1)이기 때문에, 은닉층을 여러 개 쌓으면 경사하강법 사용 시 **편향 이동(bias shift)** 현상이 생긴다.
→ 즉, 출력 값이 0 근처보다는 양수 쪽으로만 몰리게 되어 학습 효율이 떨어진다.
- 하이퍼볼릭 탄젠트 함수는 **중앙값이 0**이라서 양수/음수 모두 출력할 수 있다.
→ 그래서 **시그모이드보다 학습 효율성이 더 높다.**

- 또한, 시그모이드 함수보다 범위가 넓기 때문에 출력값의 변화폭이 더 크고, 그로 인해 **기울기 소실(Gradient Vanishing)** 증상이 더 적은 편이다.

⇒ 은닉층에서 시그모이드 함수와 같은 역할을 하는 레이어를 쌓고자 한다면, 하이퍼볼릭 탄젠트를 사용하는 것이 효과적이다.

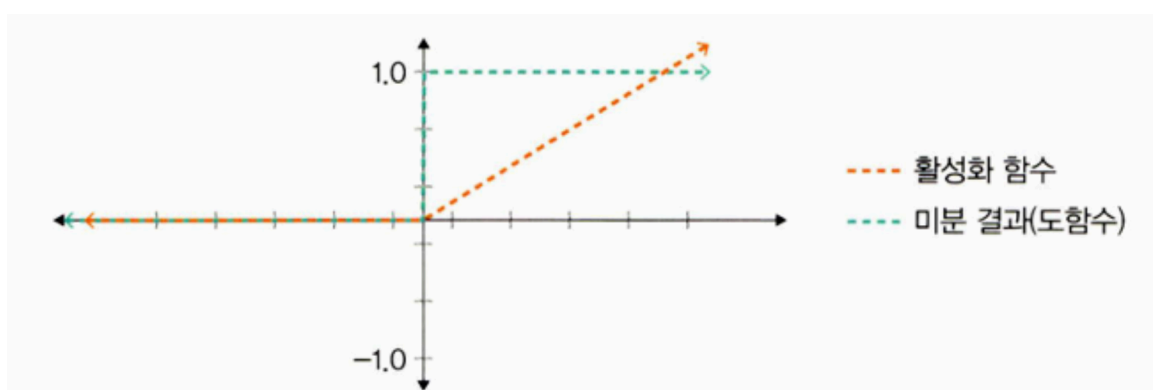
하지만 하이퍼볼릭 탄젠트 함수도 완벽한 건 아니다.

- 입력 값 $|x|$ 가 5보다 커지면, 기울기(Gradient)가 0에 가까워지면서 **기울기 소실(Gradient Vanishing)** 문제가 여전히 발생한다.
- 즉, 시그모이드보다 낫긴 하지만 **기울기 소실 문제 자체가 완전히 해결되지는 않는다.**

구분	시그모이드 함수	하이퍼볼릭 탄젠트 함수
범위	0 ~ 1	-1 ~ 1
중앙값	0.5	0
미분 최댓값	0.3	1

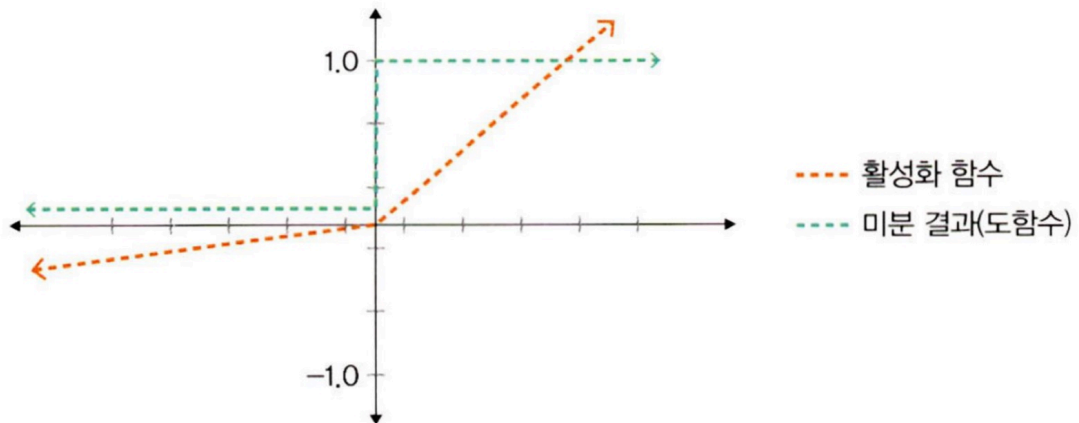
3 렐루(ReLU) 함수

- 최근 활발히 사용. 입력(x)이 음수일 때는 0출력, 양수일 때는 x 출력
- 경사하강법에 영향을 주지 X → 빠른 학습속도, 기울기 소멸 문제 X
- 일반적으로 은닉층에서 사용되며, 하이퍼볼릭 탄젠트 함수 대비 학습 속도가 6배 빠르다.
음수값을 받으면 항상 0을 출력해 학습 능력이 감소하는 문제가 있어 **리키 렐루 함수(Leaky ReLU)** 등을 사용한다.



4 리키 렐루(Leaky ReLU) 함수

- 입력 값이 음수이면 0이 아닌 0.001처럼 매우 작은 수를 반환
- 입력 값이 수렴하는 구간이 제거되어 렐루 함수를 사용할 때 생기는 문제를 해결할 수 있다.



5 소프트맥스(softmax) 함수

- 입력 값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 만들며, 가장 큰 입력값에 대해 가장 높은 확률을 부여한다.
- 딥러닝에서 출력 노드의 활성화 함수로 많이 사용

$$\sigma(z_i) = \text{softmax}(z_i) = \frac{e^{z_i}}{e^{z_1} + e^{z_2} \dots e^{z_n}} = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

- z_i : 입력 벡터 z 의 i 번째 원소 = **입력값**
- $\sum_{j=1}^n e^{z_j}$: 모든 입력 벡터 요소의 **지수값합계 분모, 정규화 역할**
- **softmax에서 자연상수를 사용하는 이유**
 1. 출력값의 합이 1이 되어, 확률적 표현이 가능하다.
(확률적이라고 명시한 이유는 이것이 정확히 확률을 의미하지는 않고 상대적 크기를 확률로 나타내는 개념)
 2. 모든 값이 양수가 되어 계산이 용이하다.

3. 미분 과정이 단순해져, 수치적으로 안정적이어서 부동소수점 연산에서 오버플로우/언더플로우 문제를 줄인다.

딥러닝(역전파)에 편미분 필요해요

미분 계산이 쉽도록 하는데 e 가 제격!
 $\therefore e^x$ 는 미분하면 자기 자신 이기 때문이죠.

$$\frac{d}{dx} e^x = e^x$$

그래서 편미분이 이렇게 간단(?) 해져요!

$$\frac{\partial \sigma(z_i)}{\partial z_j} = \sigma(z_i) (\delta_{ij} - \sigma(z_i))$$

왜 자연상수 지수함수 e^x 를 사용할까?

z_i 입력값(Logit) $-\infty \sim \infty$ $\sigma(z_i)$ 출력값 $0 \sim 1$

z_i	$\sigma(z_i) = \frac{e^{z_i}}{e^0 + e^1 + e^2}$
0	$\frac{e^0}{e^0 + e^1 + e^2} = \frac{1}{11.107} \approx 0.090$
1	$\frac{e^1}{e^0 + e^1 + e^2} = \frac{2.718}{11.107} \approx 0.245$
2	$\frac{e^2}{e^0 + e^1 + e^2} = \frac{7.389}{11.107} \approx 0.665$

1. 출력 값의 합이 1이 된다 → 확률적 표현
 2. 모든 값이 0보다 크다 → 양수
 3. e^x 는 미분하면 자기 자신인 유일한 함수
 + 지수는 큰 값과 작은 값의 차이를 강조

손실 함수

- 경사 하강법: 학습률과 손실 함수의 순간 기울기를 이용하여 가중치를 업데이트하는 방법
 = 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동시키는 방법
 이때, 오차를 구하는 방법이 바로 손실 함수

손실 함수: 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이 나는지 평가하는 지표

- 손실 함수 값이 크다 → 많이 틀렸다
- 손실 함수 값이 0에 가깝다 → 완벽하게 추정할 수 있다
- 대표적 손실 함수: 평균 제곱 오차(Mean Squared Error, MSE), 크로스 엔트로피 오차(Cross Entropy Error, CEE)

평균제곱오차(Mean Squared Error, MSE)

- MSE: 실제 값과 예측 값의 차이를 제곱하여 평균. 따라서 값이 작을수록 예측력이 좋음.
- 회귀에서 손실함수로 주로 사용됨.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

Copyright © Gilbut, Inc. All rights reserved.

```
import torch
```

```
loss_fn = torch.nn.MSELoss(reduction='sum')
```

```
y_pred = model(x)
```

```
loss = loss_fn(y_pred, y)
```

크로스 엔트로피 오차(Cross Entropy Error, CEE)

- CEE: 분류 문제에서 원-핫 인코딩을 했을 때만 사용하는 오차 계산법
- 일반적으로 분류 문제에서는 시그모이드 함수(출력 0,1 구분)를 사용하는데, 시그모이드 함수에 포함된 자연 상수 e 때문에 MSE를 적용하면 울퉁불퉁한 그래프가 출력됨 → 크로스 엔트로피 손실 함수 사용
- 하지만 경사 하강법 과정에서 학습이 지역 최소점에서 멈출 수 있음 → 이를 방지하고자 자연 상수 e에 반대되는 자연 로그를 모델의 출력 값에 취함

$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$$\left(\begin{array}{l} \hat{y}_i: \text{신경망의 출력(신경망이 추정한 값)} \\ y_i: \text{정답 레이블} \\ i: \text{데이터의 차원 개수} \end{array} \right)$$

Copyright © Gilbut, Inc. All rights reserved.

```
loss = nn.CrossEntropyLoss()
```

```
input = torch.randn(5, 6, requires_grad=True) ----- torch.randn은 평균이 0  
이고 표준편차가 1인 가우시안 정규분포를 이용하여 숫자를 생성
```

```
target = torch.empty(3, dtype=torch.long).random_(5) ----- torch.empty는  
dtype torch.float32의 랜덤한 값으로 채워진 텐서를 반환  
output = loss(input, target)  
output.backward()
```

4.2.2 딥러닝 학습

1 순전파

- **과정:** 입력 데이터 → 은닉층 → 출력층
- 각 뉴런: 이전 층에서 받은 값에 **가중합**과 **활성화 함수** 적용 → 다음 층으로 전달
- 데이터가 모든 층을 통과하고 모든 뉴런이 계산을 완료하면 최종적으로 최종 층(출력층)에 **예측 값** 도달

2 역전파



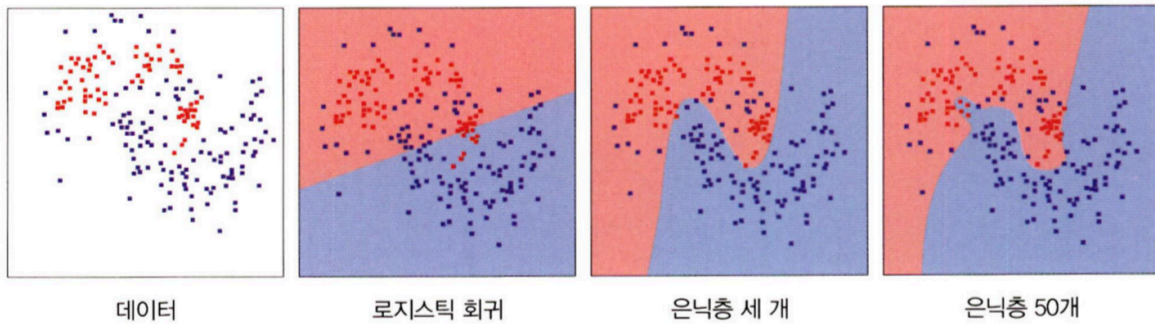
손실 함수 비용은 0이 이상적. '0'에 가깝게 하기 위해 훈련을 반복하며 가중치를 조정

- **과정:** 손실 함수로 예측 값과 실제 값의 차이(오차) 계산 → 출력층 → 은닉층 → 입력층 방향으로 역전파
- 각 뉴런의 **기여도(가중치)**에 따라 값이 달라짐
= 기존의 가중치 - [(예측값-실제값)을 각 뉴런의 가중치로 미분]
→ 각 뉴런 결과를 다시 순전파의 가중치 값으로 사용

4.2.3 딥러닝의 문제점과 해결 방안

딥러닝의 핵심: 활성화 함수가 적용된 여러 은닉층 결합으로 비선형 영역 표현하는 것

▼ 그림 4-13 은닉층이 분류에 미치는 영향



⇒ 활성화 함수가 적용된 은닉층 개수가 많을수록 데이터 분류가 잘 됨

문제점

1. 과적합 문제 발생

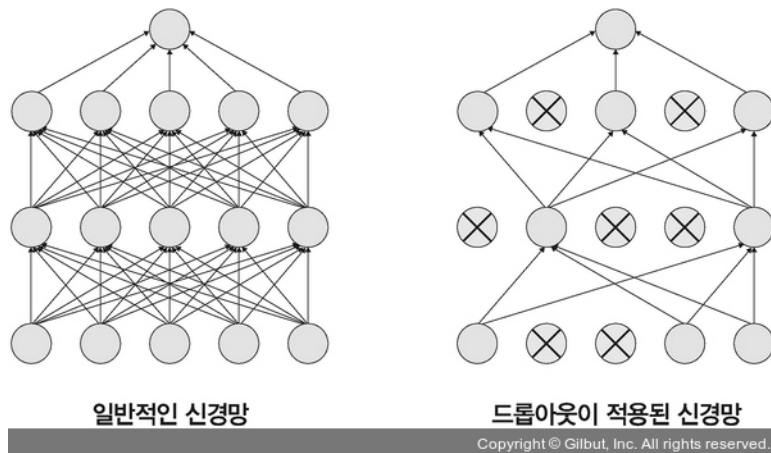
- 과적합 문제(over-fitting): 훈련 데이터를 과하게 학습할 때 발생
- 새로운 데이터(검증/테스트 데이터)에는 성능이 떨어짐(오차 증가)



⇒ 해결방안: 드롭아웃(dropout)

드롭아웃(dropout)

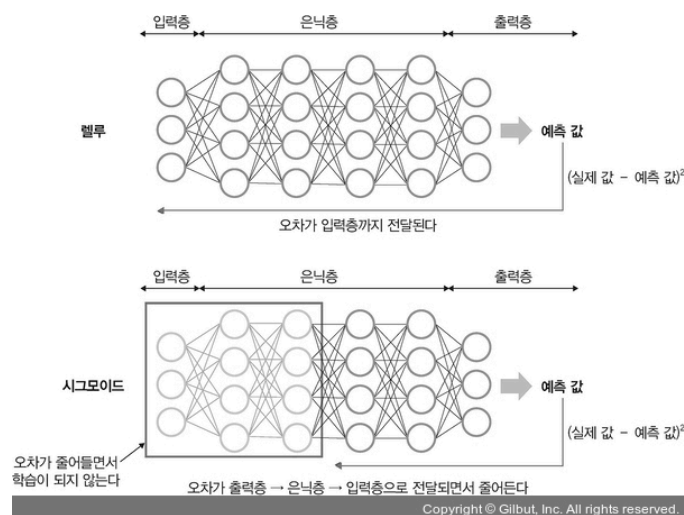
- 신경망 모델이 과적합되는 것을 피하기 위한 방법으로, 학습 과정 중 임의로 일부 노드들을 학습에서 제외시킨다.



2. 기울기 소멸 문제 발생

- 기울기 소멸 문제: 출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상
- 은닉층이 많은 신경망에서 주로 발생함.
- 기울기가 소멸 → 학습되는 양이 '0'에 가까워짐 → 학습이 더디게 진행되다 오차를 더 줄이지 못하고 그 상태로 수렴하는 현상

⇒ 해결방안: 시그모이드 or 하이퍼볼릭 탄젠트 대신 **렐루 활성화 함수 사용**



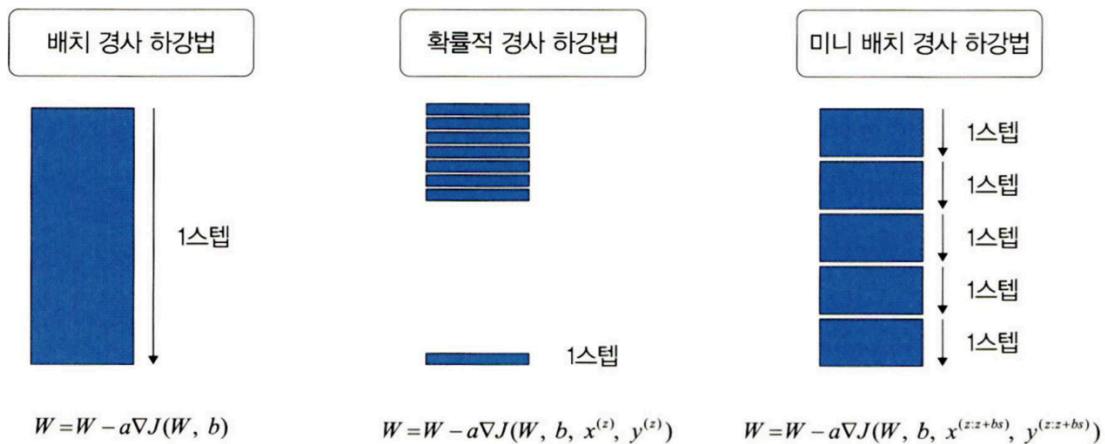
3. 성능이 나빠지는 문제 발생

- 경사하강법 과정 중 성능이 나빠지는 문제 발생

- 경사하강법은 손실함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동하는 과정을 반복하는데, 이때 local minimum 에 빠져 global minimum 을 찾지 못해 성능이 나빠지는 문제가 발생한다.

⇒ 해결방안: **확률적 경사 하강법과 미니 배치 경사 하강법 사용 + 옵티마이저**

▼ 그림 4-18 경사 하강법의 유형

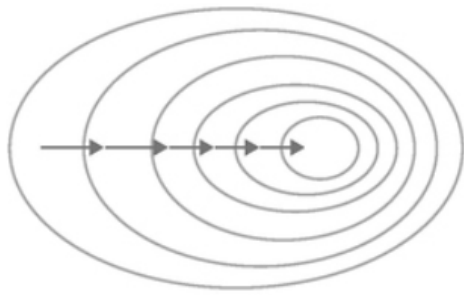


(1) 배치 경사 하강법 (BGD)

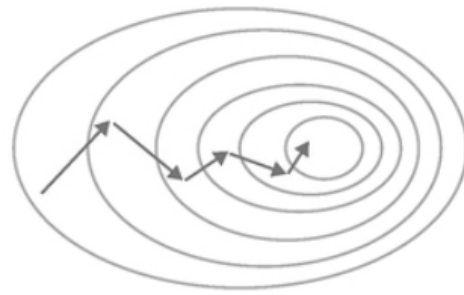
- 전체 훈련 데이터셋에 대해 가중치를 편미분 하는 방법
- 기울기를 한 번만 계산해 업데이트
- 단점: 한 스텝에 모든 훈련 데이터셋을 사용해 학습이 오래걸림

(2) 확률적 경사 하강법 (SGD)

- 배치 경사 하강법의 학습이 오래 걸리는 단점을 개선한 방법
- 임의로 선택한 데이터에 대해 기울기를 계산하는 방법
- 적은 데이터를 사용하므로 빠른 계산이 가능
- 파라미터 변경폭이 불안정하고, 때로는 정확도가 BGD 보다 낮을 수 있지만 학습 속도가 빠르다.



배치 경사 하강법



확률적 경사 하강법

(3) 미니 배치 경사 하강법(mini-batch gradient descent)

- 데이터를 미니배치 여러개로 나누고, 미니배치 한 개마다 기울기를 구한 후, 그것의 평균 기울기를 이용해 모델을 업데이트 해서 학습하는 방식
- 전체 데이터를 계산하는 것보다 빠르다.
- 확률적 경사 하강법보다 안정적이어서 실제로 가장 많이 사용한다. (속도도 빠름)

```
from torch.utils.data import Dataset
from torch.utils.data import DataLoader

class CustomDataset(Dataset) :

    def __init__(self) :
        self.x_data = [[1,2,3],[4,5,6],[7,8,9]]
        self.y_data = [[12],[18],[11]]

    def __len__(self) :
        return len(self.x_data)

    def __getitem__(self,idx) :
        x = torch.FloatTensor(self.x_data[idx])
        y = torch.FloatTensor(self.y_data[idx])
        return x, y

dataset = CustomDataset()
dataloader = DataLoader(
    dataset,
    batch_size = 2 # 미니배치크기로 2의 제공수를 사용하겠다는 의미
```

```
suffle = True # 데이터를 불러올때마다 랜덤으로 섞어 불러옴
)
```

(4) 옵티마이저 (Optimizer)

- 확률적 경사하강법의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 학습 속도와 운동량을 조절하는 optimizer 를 적용해볼 수 있다.

✓ 속도 조정

- Adagrad, Adadelata, RMSProp

✓ 운동량 조정

- Momentum, NAG

✓ 속도와 운동량 혼용

- Adam

4.2.4 딥러닝을 사용할 때 이점

1 특성 추출

- 사람이 직접 feature 를 정의해주어야 하는 SVM, 나이브 베이즈, 로지스틱 회귀와 같은 전통적인 머신러닝 방법론과 달리 딥러닝에서는 스스로 특성을 추출해낸다.

2 빅데이터의 효율적 활용

- 딥러닝은 데이터 사례가 많을 수록 성능이 향상된다.
- 반대로 확보된 데이터가 적다면 딥러닝의 성능 향상을 기대하기 어려우므로 머신러닝을 고려해야 한다.

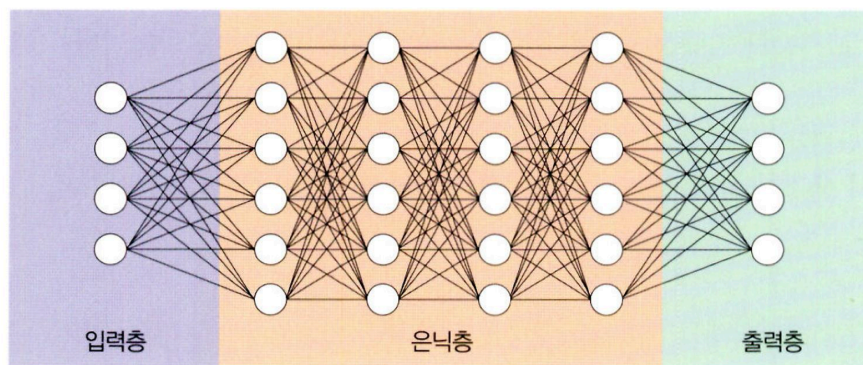
4.3 딥러닝 알고리즘

4.3.1 심층 신경망(DNN)

- 입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망
- 머신러닝에서 비선형 분류를 하기 위해 여러 트릭을 썼던 것과 달리, 심층 신경망은 다수의 은닉층을 추가했기 때문에 별도의 트릭 없이 비선형 분류가 가능함.
- 장점: 다양한 비선형적 관계를 학습할 수 있음
- 단점: 연산량이 많고 기울기 소멸 문제가 발생

→ 드롭아웃, 렐루 함수, 배치 정규화 등을 적용한다.

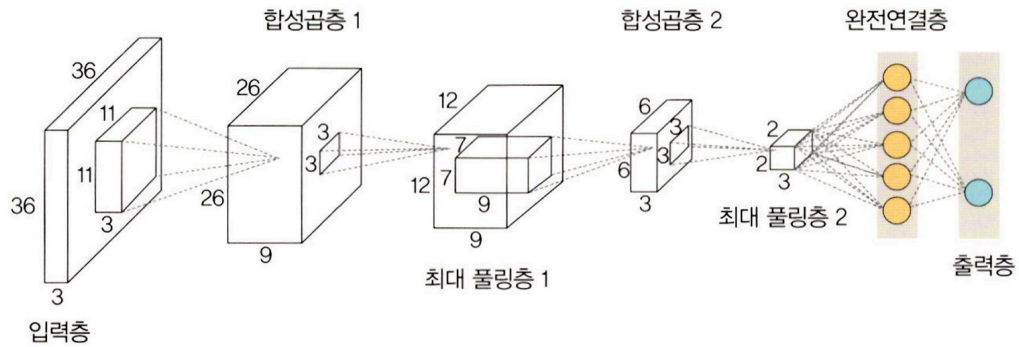
♥ 그림 4-24 심층 신경망



4.3.2 합성곱 신경망(CNN)

- 합성곱 신경망(Convolutional Neural Network, CNN): 합성곱층과 풀링층을 포함하는 (Convolutional layer + Pooling layer) 이미지 처리 성능이 좋은 인공 신경망 알고리즘
- 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는 데 유용한 신경망임.

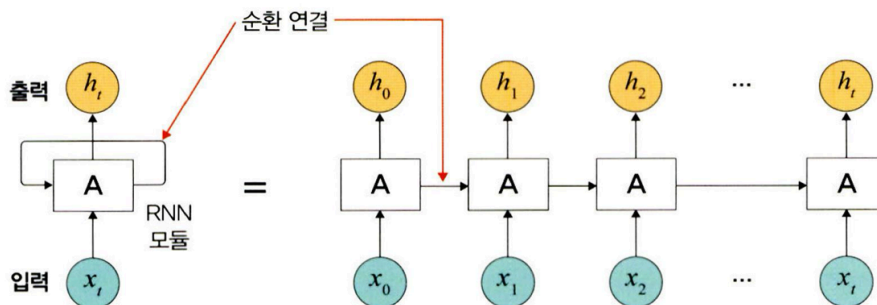
▼ 그림 4-25 합성곱 신경망



4.3.3 순환 신경망(RNN)

- 순환 신경망(Recurrent Neural Network, RNN): 시계열 데이터(음악, 영상 등) 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망.
- 순환 = 자기 자신을 참조 = 현재 결과가 이전 결과와 연관이 있다는 의미
- 데이터는 동적이고 길이가 가변적이다.
- 자연어 처리 분야와 궁합이 맞음.
 - 언어모델링, 텍스트 생성, 자동번역, 음성인식, 이미지 캡션 생성 등 다양한 예시

▼ 그림 4-26 순환 신경망



순환 신경망의 특징은 다음과 같습니다.

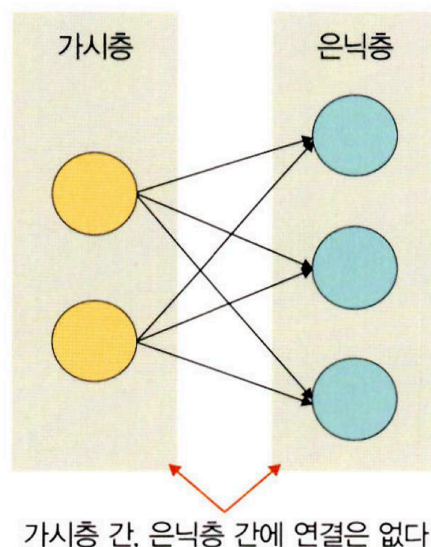
- 시간성(temporal property)을 가진 데이터가 많습니다.
- 시간성 정보를 이용하여 데이터의 특징을 잘 다룹니다.
- 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적입니다.
- 매우 긴 데이터를 처리하는 연구가 활발히 진행되고 있습니다.

- 기울기 소멸 문제로 학습이 제대로 되지 않는 문제가 있다.
- → 이를 해결하고자 메모리 개념을 도입한 LSTM(Long-Short Term Memory)이 순환 신경망에서 많이 사용된다.

4.3.4 제한된 볼츠만 머신

- 가시층(visible layer)과 은닉층(hidden layer)으로 구성된 모델
- 각 층은 독립적으로 구성되어 있다. (가시층은 은닉층과만 연결됨 / 은닉층-은닉층 연결은 X)
-
- 딥러닝에서 많이 사용되는 알고리즘은 CNN, RNN이라 제한된 볼츠만 머신과 심층 신뢰 신경망은 상대적으로 많이 사용되지 않는다.

▼ 그림 4-27 제한된 볼츠만 머신



제한된 볼츠만 머신의 특징은 다음과 같습니다.

- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링(collaborative filtering), 특성 값 학습(feature learning), 주제 모델링(topic modelling)에 사용합니다.
- 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능합니다.
- 심층 신뢰 신경망(DBN)의 요소로 활용됩니다.

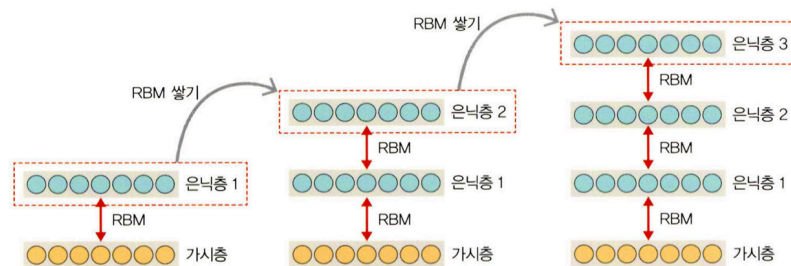
4.3.5 심층 신뢰 신경망

- 입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망

= 사전 훈련된 제한된 볼츠만 머신을 쌓아 올린 구조 → 레이블이 없는 데이터에 대한 **비지도 학습**이 가능하다.

- 계층적 구조 형성
- 위로 올라갈수록 추상적 특성을 추출한다.

♥ 그림 4-28 심층 신뢰 신경망



심층 신뢰 신경망의 특징은 다음과 같습니다.

- 순차적으로 심층 신뢰 신경망을 학습시켜 가면서 계층적 구조를 생성합니다.
- 비지도 학습으로 학습합니다.
- 위로 올라갈수록 추상적 특성을 추출합니다.
- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용합니다.

4.4 우리는 무엇을 배워야 할까?

머신러닝과 딥러닝은 우리가 얻고 싶은 결과를 도출하기 위한 도구이다. 어떤 도구를 선택했을 때 더 좋은 성능과 효과를 얻을지는 데이터를 수집하고 분석하는 사람만이 알 수 있다.

- 간단한 선형 회귀 분류 → 머신러닝만으로도 충분
- 복잡한 비선형 데이터 분류 → 딥러닝 학습 필요

⇒ 데이터를 활용하여 얻고자 하는 것에 따라 머신러닝이나 딥러닝을 선택해서 학습하고 데이터를 훈련시키면 된다.